SpMV analysis

Andreas Chini
[258909]
andreas.chini@studenti.unitn.it

***Abstract*—An analysis on different SpMV (Sparse matrix vector multiplication) kernels on the COO and CSR sparse matrix formats. The analysis will not only look at the performance but also comment on the results to understand them.**

I. Introduction

Sparse Matrix-Vector Multiplication (SpMV) is a critical computational bottleneck in scientific computing and machine learning. Because it performs only two operations per non-zero element ($2 \times$ nnz), it is heavily memory-bound, limited by bandwidth, irregular memory access, and thread load imbalances. Optimizing SpMV requires moving beyond basic format comparisons to align storage layouts directly with matrix topology. Standard formats like COO and CSR fluctuate drastically in efficiency depending on a matrix's structural regularity. This investigation addresses a core question: How can custom CUDA kernels be designed to minimize global memory overhead and balance severe row irregularities? By evaluating custom kernels against a CPU baseline and NVIDIA's cuSparse, this study analyzes how matrix traits dictate performance limits on modern GPUs

II. Methodology

A. Matrix formats used

The sparse matrix formats used in this analysis are the COO format and the CSR format. The first is used mostly for storage purposes and the second is used mostly for arithmetic operations. We will see how they perform with different types of matrices and different kernels.

B. Different implementations

In this analysis different kernels will be used, we will first start with a CPU kernel (both serial and parallel) of a basic COO and CSR SpMV implementation, then those base kernels will be implemented on the GPU side. We will then have a custom built kernel for COO SpMV, and two custom kernels for CSR SpMV, then for comparison there will be a cuSparse SpMV that will serve as a roof for the best possible performance one can achieve.

C. Validation method

To validate numerical correctness, the GPU outputs are compared against the serial CPU baseline using a mixed-tolerance check (abs_tol $= 10^{-3}$, rel_tol $= 10^{-3}$). While most matrices pass perfectly, highly irregular matrices with massive rows—like FullChip—frequently show high localized errors of 1% to 2% (e.g., 1123.87 on CPU vs 1099.78 on the GPU custom kernel). Because floating-point addition is non-associative, these mismatches are the result of changing the summation order. In parallel chunk-based reductions or asynchronous atomic operations, there are thousends of such cases. This discrepancy is an intrinsic artifact of parallel hardware acceleration, to change it more precision is needed, but that also doubles the complexity and halves the computation speed.

D. Measurement methodology

To extract the kernel time two different functions will be used, clockgettime() in the CPU case and cudaEvent() in the GPU case. In the case of the Gflops/s the standard 2*nnz floating point operations per matrix-vector multiplication will be used. To get consistent results only the edu01 queue was used.

III. Dataset analysis

The code for the SpMV and the matrices links for the download can be found at the following link: [https://github.com/AndreasPr98/GPU_computing_deliverable.git]

A. Input vector

The input vector is a float vector with random values between 1 and 100.

B. Sparse Matrices

TABLE I
METRICS OF THE EXAMINED SPARSE MATRICES

Matrix Name	Rows	Cols	NNZ	Sparsity	Max nnz/row	Min nnz/row	Avg row length	Row variance
bone010	986,703	986,703	71,666,325	99.9926%	81	12	72.63	249.97
ASIC_680ks	682,712	682,712	2,329,176	99.9995%	210	1	3.41	18.09
boyd2	466,316	466,316	1,500,397	99.9993%	93,262	2	3.22	37,990.72
FullChip	2,987,012	2,987,012	26,621,990	99.9997%	2,312,481	1	8.91	3,264,522.73
Ga41As41H72	268,096	268,096	18,488,476	99.9743%	702	18	68.96	11,106.54
ldoor	952,203	952,203	46,522,475	99.9949%	77	28	48.86	142.72
mc2depi	525,825	525,825	2,100,225	99.9992%	4	2	3.99	0.01
rajat31	4,690,002	4,690,002	20,316,253	99.9999%	1,252	1	4.33	1.22
Si41Ge41H72	185,639	185,639	15,011,265	99.9564%	662	13	80.86	16,121.85
StocF-1465	1,465,137	1,465,137	21,005,389	99.9990%	189	1	14.34	6.62
webbase-1M	1,000,005	1,000,005	3,105,536	99.9997%	4,700	1	3.11	642.38

As it can be observed from table I there are different matrices, most of them were taken from the paper by Chu et al. [1], and some were randomly taken from the Suite Sparse matrix collection [2]. What they share in common is the high sparsity ($> 99.9\%$), but there are different types of challenges in those matrices. For example there are matrices with short rows (like webbase-1M) that will slow down computation because of the result of an SpMV changes position in the array every time the row changes, then there are highly irregular matrices (like FullChip) that require to create a kernel that works good both with extremely short and extremely long rows, there are also very regular matrices (like mc2depi) on which the kernel architecture is very easy to build.

IV. Kernels pseudo-code

A. Base kernels

The algorithm 1 explains the base idea that was implemented both in CPU and GPU for the COO basic kernel, on the device side atomicAdd is necessary to provide an error-free addition to the result vector. This algorithm works by assigning to each thread (i) only one multiplication, and the result is stored in the result vector directly, as said before, to avoid conflict from different threads that sum results on the same value, atomicAdd is necessary, but this function serializes the writing process, stopping the thread up until completion, this can heavily slow down the process.

Algorithm 1 Basic COO Kernel

```
1:  procedure COO_BASE_KERNEL(nnz, row_indices,
                              col_indices, values, x, y)
2:  i ← blockIdx.x × blockDim.x + threadIdx.x
3:  if i < nnz then
4:      row ← row_indices[i]
5:      col ← col_indices[i]
6:      val ← values[i]
7:      prod ← val × x[col]
8:      atomicAdd(&y[row], prod)
9:  end if
10: end procedure
```

In the CSR kernel (Algorithm 2) the operation is different, here each thread computes one entire row, since there is no conflict when saving the results there is no need for atomicAdd, and this kernel works better than its COO counterpart, but if the matrix is highly irregular with some very long rows, the threads assigned on the long rows will be stuck computing while all the other will have already finished, so it's not a perfect solution for the SpMV problem.

Algorithm 2 Basic CSR Kernel

```
1:  procedure CSR_BASE_KERNEL(n_rows, row_offsets,
                              col_indices, values, x, y)
2:  row ← blockIdx.x × blockDim.x + threadIdx.x
3:  if row < n_rows then
4:      sum ← 0.0
5:      start ← row_offsets[row]
6:      end ← row_offsets[row + 1]
7:      for k ← start to end − 1 do
8:          col ← col_indices[k]
9:          sum ← sum + values[k] × x[col]
10:     end for
11:     y[row] ← sum
12: end if
13: end procedure
```

B. Custom COO kernel

Although COO format is mainly used for storage and rarely for computation, a kernel was still implemented to see if we could get better performance than the base kernel implementation. The idea is to try to reduce the burden on the atomicAdd function that is probably the main bottleneck and use some cache to speed up the whole process.

As it can be seen in the algorithm n°3 we still start with each thread computing each multiplication (row 8) but after that we use the threads registers commands to do a reduction (while cycle row 11) of all the previous threads that have the same row index, using this we reduce with a complexity of log2(x), then at completion of this step we find out if the thread is an end thread (the one with the reduction result) (row 19). If it is the end thread we use atomicAdd to write the result in the output vector, we are still forced to use atomic because there will be lines that are longer than the block of threads with the same row index in the same warp, and we cannot use register operations in between different warps.

With this setup we should have the advantage of much less use of atomic, and the greatly increased speed of registers memory for the reduction operation.

Algorithm 3 Custom COO Kernel

```
1:  procedure COO_C_KERNEL(nnz, row_indices, col_indices,
                           values, x, y)
2:  i ← blockIdx.x × blockDim.x + threadIdx.x
3:  lane_id ← threadIdx.x%32
4:  my_row ← −1
5:  my_val ← 0.0
6:  if i < nnz then
7:      my_row ← row_indices[i]
8:      my_val ← values[i] × x[col_indices[i]]
9:  end if
10: offset ← 1
11: while offset < 32 do
12:     src_row ← shuffle_up(my_row, offset)
13:     src_val ← shuffle_up(my_val, offset)
14:     if lane_id ≥ offset and src_row = my_row then
15:         my_val ← my_val + src_val
16:     end if
17:     offset ← offset × 2
18: end while
19: next_row ← shuffle_down(my_row, 1)
20: is_tail ← (lane_id = 31) or (next_row ≠
    my_row) or (i = nnz − 1)
21: if i < nnz and is_tail then
22:     atomicAdd(y[my_row], my_val)
23: end if
24: end procedure
```

C. Custom CSR kernel 1

The first custom CSR kernel (algorithm n° 4) uses a similar strategy to the custom COO kernel (algorithm n° 3), use the register's fast memory to perform reduction, but if the COO version had different rows in the same warp the CSR version that launches an entire warp per row can benefit from having

the certainty of all the warp processing the same row, so a complete 32 to 1 reduction with no checks in between.

If we compare it to the algorithm n°2 we can see that we still have the 1 row per compute unit (row 2) that we had there, but this time it's a warp_id and not a thread_id, and as mentioned before it can be seen the warp reduction algorithm (row 14) that performs the cumulative sum with the register's function. In the end the thread 0 of the warp is the one that performs the writing on the output vector (row 18).

Algorithm 4 Custom CSR Kernel 1

```
1:  procedure CSR_C_1_KERNEL(n_rows, row_ptr,
                             col_indices, values, x, y)
2:  warp_id ← ⌊(blockIdx.x × blockDim.x + threadIdx.x)/32⌋
3:  lane ← threadIdx.x mod 32
4:  if warp_id < n_rows then
5:      row_start ← row_ptr[warp_id]
6:      row_end ← row_ptr[warp_id + 1]
7:      sum ← 0.0
8:      i ← row_start + lane
9:      while i < row_end do
10:         sum ← sum + values[i] × x[col_indices[i]]
11:         i ← i + 32
12:     end while
13:     offset ← 16
14:     while offset > 0 do
15:         sum ← sum + shuffle_down(sum, offset)
16:         offset ← ⌊offset/2⌋
17:     end while
18:     if lane = 0 then
19:         y[warp_id] ← sum
20:     end if
21: end if
22: end procedure
```

D. Custom CSR kernel 2

This kernel (algorithm n° 6) is a little bit more structured then the previous ones, while the others just computed the results, this one starts by mapping a structure of the matrix that will be used by the main kernel to compute the output vector much faster by knowing already were to go to find the data. It's a common practice that is also used by other SpMV algorithms like cuSparse, while it may take some time to compute the map (in this case 2 to 3 times the SpMV computation time), the computation is sped up, and since in many applications the matrix stays the same and only the vector changes, then the time spent on mapping the matrix doesn't impact much on the final results since it's only done once.

In the preparation algorithms (n° 5) there is a first one (first paragraph) that simply counts how many chunks of data are there inside each row of the matrix, then on the host side a parallel reduction will be executed so to have the starting chunk of each row. The second preparation algorithm (second paragraph) takes the starting chunk of each row, and outputs a row for each chunk_id and a chunk offset for each chunk_id, so each chunk_id now know what row it's assigned and where to start inside that row.

Algorithm 5 Preparation Kernels for CSR Custom kernel 2

```
1:  procedure CHUNK_COUNT_KERNEL(n_rows, row_ptr,
                                 chunk_counts, values, x, y)
2:  row ← blockIdx.x × blockDim.x + threadIdx.x
3:  if row < n_rows then
4:      row_len ← row_ptr[row + 1] − row_ptr[row]
5:      chunk_counts[row] ← ⌊(row_len + CHUNK_SIZE −
    1)/CHUNK_SIZE⌋
6:  end if
7:  end procedure
```

```
1:  procedure CHUNK_MAP_KERNEL(n_rows, row_ptr,
                               chunk_offsets, chunk_rows, chunk_starts)
2:  row ← blockIdx.x × blockDim.x + threadIdx.x
3:  if row < n_rows then
4:      row_len ← row_ptr[row + 1] − row_ptr[row]
5:      row_chunks ← ⌊(row_len + CHUNK_SIZE −
    1)/CHUNK_SIZE⌋
6:      first_chunk ← chunk_offsets[row]
7:      for chunk ← 0 to row_chunks − 1 do
8:          chunk_id ← first_chunk + chunk
9:          chunk_rows[chunk_id] ← row
10:         chunk_starts[chunk_id] ← chunk × CHUNK_SIZE
11:     end for
12: end if
13: end procedure
```

Now that we have a map for the row and starting point for each compute unit, the algorithm behind the second custom kernel (n° 6) for CSR SpMV it's not too complicated. We start by finding the compute unit (block of threads) chunk_id (row 3), and the number of each thread inside the compute unit (row 4). Now we calculate the starting and ending point for each c.u. and then do the multiplication with each thread by himself (row 14), then we use the register's function to reduce the c.u. results to the first thread. In the end we write the result in the output vector, if the row is shorter than the chunk size we don't use atomicAdd since there cannot be any conflict because there would be the only one c.u. to act on that row, that can speed up the kernel a little bit more.

E. '__restrict__' usage

One thing that every GPU kernel created for this analysis shares is the use of the "__restrict__" qualifier when referring to the global memory variables in the header of the function. This qualifier it's a promise you make to the compiler that no other function will change the value, it's a dangerous tool because if you actually change the value, the compiler at run time will not check if the value has changed or is being changed, and that can can lead to the program crashing if something happens. On the other hand with this promise the compiler can be much more targeted on how to treat the data,

Algorithm 6 Custom CSR Kernel 2

```
procedure CSR_C_2_KERNEL(total_chunks, row_ptr,
        col_indices, values, chunk_rows, chunk_starts, x, y)
global_thread_id ← blockIdx.x × blockDim.x + threadIdx.x
chunk_id ← ⌊global_thread_id/V⌋
vector_lane ← threadIdx.x mod V
if chunk_id < total_chunks then
    row ← chunk_rows[chunk_id]
    row_start ← row_ptr[row]
    row_end ← row_ptr[row + 1]
    row_len ← row_end − row_start
    chunk_start ← row_start + chunk_starts[chunk_id]
    chunk_end ← min(chunk_start + CHUNK_SIZE, row_end)
    sum ← 0.0
    i ← chunk_start + vector_lane
    while i < chunk_end do
        sum ← sum + values[i] × x[col_indices[i]]
        i ← i + V
    end while
    offset ← ⌊V/2⌋
    while offset > 0 do
        sum ← sum + shuffle_down(sum, offset)
        offset ← ⌊offset/2⌋
    end while
    if vector_lane = 0 then
        if row_len ≤ CHUNK_SIZE then
            y[row] ← sum
        else
            atomicAdd(y[row], sum)
        end if
    end if
end if
end procedure
```

in the case of a vector that only needs to read that means that the compiler will automatically place it in a shared read-only cache that is much faster than the global memory.

I tried to add values to the cache manually with __ldg(x) memory function, but this didn't change the run time at all, so there is a high probability that the compiler already did that on its own.

V. Results

The results of the analysis can be seen in Tab. II and Tab. III, the meaning of the data will be discussed in the next section. The data is the average of 10 complete runs, each run repeats each matrix 10 times, that means that these values are a 100 SpMV average, in the tables we can also observe the standard deviation for each matrix for each value. Interesting values apart from kernel times can be the time to pass from COO to CSR format, and the time for the second kernel to map the matrix that is going to compute.

VI. Results discussion

A. CPU results

As it can be seen both in Fig. 1 and Fig. 2 the CPU performance as expected doesn't even come near to the GPU performance. SpMV operations are notoriously a GPU operation, and why can see why from the graphs, but having a CPU baseline can be useful for comparison.

B. CuSparse results

If we again take a look at Fig. 1 and Fig. 2 we can see that the best performer is always cuSparse, and even tho sometimes the custom kernels almost match the performance they never beat it. This is because cuSparse (as CSR custom 2) does an analysis of the matrix before computation, so it can decide the best way to solve the problem, it is also one of the best libraries to do SpMV so it's hard to beat.

That said it is important to have an empirical roof of what the best performance can be so to see if one has done a good job with the kernels or it can better the algorithm.

C. COO results

The custom 1 algorithm performs fairly well across the board, staying most of the time at the 80% of cuSparse threshold, the same cannot be said for the basic kernel that sometimes has optimal performance and sometimes is a little as 5% of the cuSparse results. If we take a look at Tab. I we can see that most of the matrices where the performance is very bad are those where the average row length is very high (e.g. bone010, ldoor, Ga41As41H72, Si41Ge41H72, ecc). SO we can speculate that the reason for this poor performance is the fact that if a lot of operation are in the same row there will be a lot of collisions in the saving of the results so atomicAdd will have to work a lot and the peformance suffers tremendously.

The custom COO kernel instead, since it uses registers, doesn't suffer from this problem, the only matrix where a performance drop can be noticed is the FullChip matrix, probably becouse it has an enormous longest row so atomic has to still do a lot of work in that case, but the performance is still very acceptable at more than 50% of the cuSparse result.

D. CSR results

In the case of the base CSR implementation the case differs greatly from the COO version, it performs very well in a lot of cases, but performs very poorly in some others. The reason for this poor performance is obvious in this case, the algorithm uses a 1 row per thread technique, so if the matrix has one (or more) extremely long row (e.g. FullChip) one thread will be forced to compute an enormous amount of data alone, and the result can be seen clearly in Fig. 2. So it can be used in specific cases but it isn't a one for all solver as we were trying to achieve.

A less obvious but similar story impacts the first CSR custom kernel, as the base kernel the performance on FullChip is terrible, and even tho it performs much better it is still not an acceptable result, furthermore this kernel suffers from another great problem, if the rows are very short (< 32) a portion of

TABLE II
COO FORMAT RESULTS

Name	Dim	Parsing time	CPU	CPU	CPU parallel	CPU parallel	Cuda base	Cuda base	Cuda Custom	Cuda Custom	Cuda cuSparse	Cuda cuSparse	COO to CSR
(unit)	(MB)	(ms)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)
ASIC-680ks	33	724.66	3109.7	1.501	3962.91	1.176	114.6	40.68	67.3	69.23	57.12	81.62	8501
std-dev	0	4.92	144.1	0.07	55.92	0.017	3.31	1.24	1.12	1.19	1.64	2.49	484
bone010	868	13335.97	136248.6	1.052	122291.84	1.166	10908.84	13.29	1428.37	101.03	1076.08	133.23	249,202
std-dev	0	74.42	498.3	0.004	1773.31	0.017	1140.88	1.58	126.79	8.42	18.58	2.26	11,720
boyd2	22	224.08	1925.0	1.56	2770.42	1.097	426.83	7.03	39.87	75.26	31.46	95.38	5885
std_dev	0	2.91	54.0	0.044	378.82	0.111	0.66	0.01	0.2	0.38	0.27	0.83	264
FullChip	343	8569.06	42895.2	1.242	49400.31	1.078	13959.21	3.85	868.75	61.84	511.39	104.16	81,939
std-dev	0	51.17	683.7	0.02	925.36	0.02	1391.7	0.37	89.66	5.88	11.68	2.32	3240
Ga41As41H72	224	4166.78	38340.0	0.965	31672.92	1.168	3469.06	10.69	424.8	87.18	299.02	123.62	64,238
std-dev	0	25.48	305.8	0.008	396.88	0.015	191.34	0.64	17.19	3.75	3.27	1.36	1425
ldoor	566	9656.95	85879.1	1.084	80184.18	1.161	60135.55	15.52	1017.43	91.69	735.94	126.47	155,865
std-dev	0	31.19	1017.9	0.013	1111.51	0.016	354.67	0.96	54.14	5.01	13.74	2.39	4451
mc2depi	29	477.37	2246.9	1.871	3503.82	1.199	50.42	83.31	52.37	80.2	39.09	107.46	6879
std-dev	0	5.11	56.9	0.046	53.74	0.018	0.23	0.37	0.13	0.2	0.25	0.7	412
rajat31	281	6341.3	25196.4	1.613	35070.91	1.159	627.91	64.89	518.91	78.31	446.59	91.01	59,754
std-dev	0	27.09	499.4	0.032	423.93	0.014	33.28	3.72	5.54	0.85	8.13	1.7	3893
Si41Ge41H72	182	3398.88	31968.6	0.939	25730.04	1.167	3300.85	9.13	345.05	87.22	244.89	122.62	53,233
std-dev	0	14.59	128.3	0.004	309.36	0.014	205.62	0.62	17.05	4.65	3.58	1.8	1866
StocF-1465	264	4262.35	28557.0	1.471	36315.01	1.157	1428.76	29.53	493.71	85.27	380.27	110.49	63,655
std-dev	0	30.04	307.2	0.016	506.45	0.016	94.8	2.08	23.4	4.2	4.8	1.41	1806
webbase-1M	45	853.92	4789.4	1.297	5307.49	1.17	302.36	20.76	84.95	73.36	76.21	81.76	11,172
std-dev	0	9.94	112.1	0.03	55.61	0.012	32.7	2.22	5.12	4.34	4.59	4.85	506

TABLE III
CSR FORMAT RESULTS

Name	Dim	CPU	CPU	CPU parallel	CPU parallel	Cuda base	Cuda base	Cuda Custom 1	Cuda Custom 1	Cuda Custom 2	Cuda Custom 2	Cuda cuSparse	Cuda cuSparse	Total time
(unit)	(MB)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)	(Gflops/s)	(us)	(Gflops/s)	(ms)
ASIC-680ks.mtx	2,851	1.642	627.86	7.446	90.04	51.766	494.34	9.431	126.53	36.84	915.8	54.91	84.9	1,322.20
std-dev	212	0.115	39.9	0.455	2.14	1.3	14.22	0.289	3.31	1.02	586.5	1.46	2.39	107.2
bone010.mtx	64,118	2.236	15,045.76	9.736	1,376.14	104.246	1,101.48	130.39	824.92	173.8	695.3	695.04	206.22	18,379.20
std-dev	725	0.025	2,422.09	1.435	43.51	3.15	54.8	5.811	14.33	2.88	49.8	0.38	0.11	201.9
boyd2.mtx	1,353	2.218	459.3	6.535	6,116.90	0.491	432.71	6.935	53.58	56.0	921.6	27.83	107.82	664.6
std_dev	34	0.054	8.1	0.114	36.85	0.003	1.62	0.026	0.22	0.23	1,254.10	0.25	0.99	51.9
FullChip.mtx	31,029	1.717	13,240.60	4.04	198,865.07	0.268	9,510.13	5.599	608.42	87.51	1,222.90	360.37	147.75	13,529.00
std-dev	892	0.05	934.39	0.285	769.78	0.001	80.13	0.047	1.2	0.17	24.5	0.28	0.12	159.1
Ga41As41H72.mtx	19,140	1.932	4,495.15	8.229	516.18	71.798	316.4	117.018	246.27	150.31	4,248.10	212.08	174.55	5,827.70
std-dev	311	0.031	91.75	0.168	24.96	3.684	11.55	4.509	8.32	5.29	6,711.80	7.29	6.21	102.5
ldoor.mtx	43,400	2.145	9,485.44	9.819	978.72	95.308	928.14	101.39	585.84	159.41	1,252.80	491.54	190.29	13,041.80
std-dev	1,216	0.06	316.96	0.334	51.4	5.04	100.68	11.605	37.24	10.16	1,885.20	38.29	14.14	134.5
mc2depi.mtx	1,598	2.63	424.79	9.891	35.29	119.025	282.43	14.873	59.31	70.83	618.5	39.35	106.75	836.8
std-dev	40	0.067	7.41	0.173	0.12	0.416	1.27	0.067	0.25	0.3	47.3	0.18	0.5	52.5
rajat31.mtx	28,080	1.954	5,590.46	7.322	349.59	116.239	3,337.80	12.197	695.78	58.52	1,741.90	360.78	112.78	8,081.20
std-dev	538	0.05	512.67	0.646	3.6	1.214	149.12	0.586	32.82	2.94	2,820.90	13.85	4.57	144.6
Si41Ge41H72.mtx	15,713	1.911	3,496.16	8.589	444.17	67.763	242.25	124.119	202.13	148.75	1,574.60	173.46	173.3	4,833.90
std-dev	190	0.023	53.44	0.131	22.57	3.714	9.57	5.202	7.96	6.21	2,651.20	6.2	6.52	112.9
StocF-1465.mtx	19,369	2.17	4,656.26	9.082	401.58	105.25	1,059.21	39.791	409.4	102.8	715.4	277.43	151.64	6,272.60
std-dev	496	0.056	424.67	0.724	31.64	8.882	61.39	2.442	17.84	4.67	50.2	10.72	6.08	136.4
webbase-1M.mtx	4,306	1.444	1,402.16	4.432	593.22	10.539	643.16	9.738	141.14	44.28	695.1	59.92	104.58	1,403.80
std-dev	125	0.042	32.76	0.104	50.74	0.886	61.99	0.922	11.87	3.66	52.1	5.99	10.21	63.9

the threads will be wasted doing nothing, so overall this kernel is only good for regular matrices with fairly long rows.

The best result when it comes to custom kernels can be seen in the second CSR custom kernel, and even tho it still doesn't match cuSparse performance the results across the board are all acceptable. The probable reason for this result is the fact that this kernel solves a lot of problem of the other custom algorithms, it first uses registers for the reduction, second it uses multiple threads per row, and the longer the row the more threads it uses, so it adapts. And last it uses a fraction of the warp (4 or 8 threads) to compute the short rows, so it doesn't lose too much performance when the rows are short. It suffers the initial cost (2 to 3 times the run time) of mapping the matrix, but as we said before a matrix is usually used a many times unchanged so the mapping cost is only paid once.

E. COO vs CSR

Observing the Fig. 3 it can be seen that most of the times that the CSR format achieves better performance than then COO format, and while there may be some exceptions CSR is the best format for arithmetic operation. This result was very predictable since the SpMV operation is memory bound and the CSR format uses less memory for storing the row, that permits a faster data access and so a faster computation in the end. Both of the custom kernels do an acceptable job in the matrix vector multiplication operation and do so with very different matrices.

F. Conclusions

The main goal of this analysis was to create a custom kernel for SpMV operations, try to make it as fast as possible, when you hit the limit understand why is that the case, how that kernel perform in different situations and try to interpret the results to get an explanation. In the Results discussion we talked about why the kernels worked well when intended and when they didn't get to the expectations why that was the case, the main lessons we can take away from this are the fact that SpMV is still memory speed limited, but even more the fact that you don't need to spend thousands of hours on a GPU application to achieve reasonable results. And if in this case there are already programs that do it better (cuSparse) there are many niche cases where there is not premade function and it's worth to program the solution yourself on the GPU directly and achieve good performance. The main limitation as for now is probably knowledge, because the more practice and research i did the better the program became,

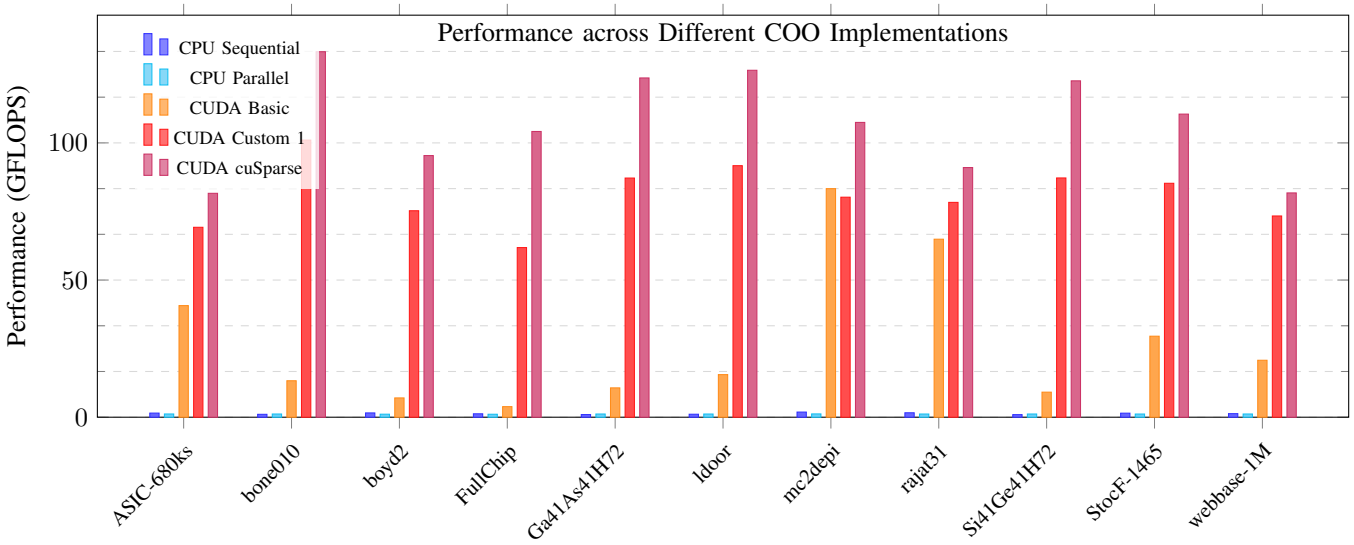

Fig. 1. GFLOPS comparison between serial, parallel CPU, and various GPU accelerated implementations.

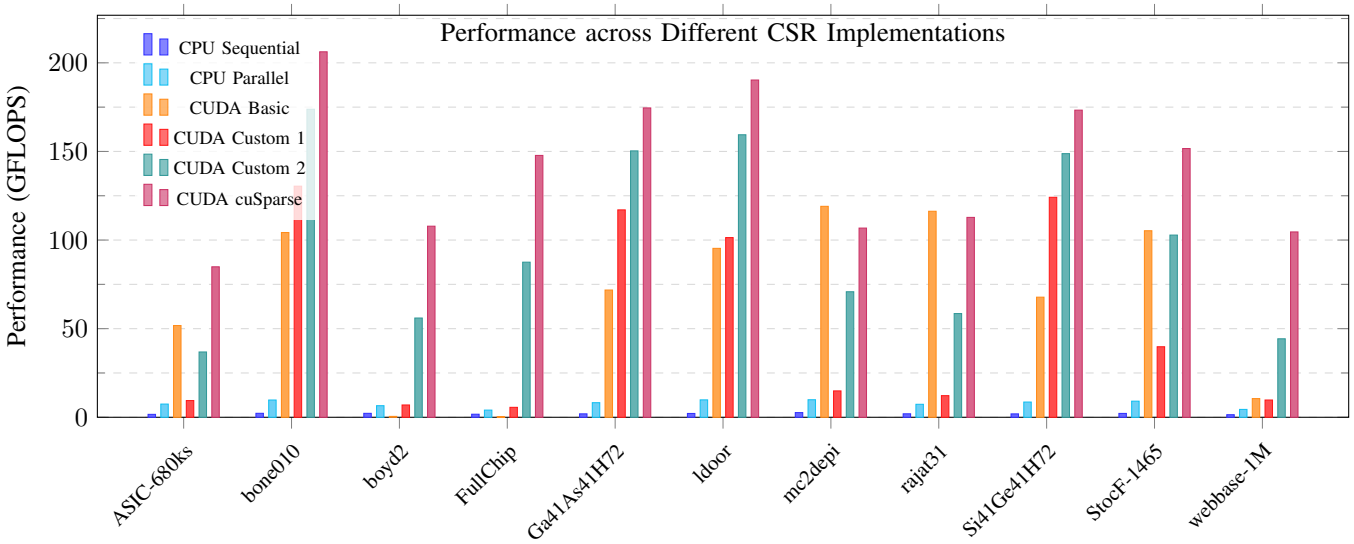

Fig. 2. GFLOPS comparison between serial, parallel CPU, and various GPU accelerated implementations using the CSR sparse storage layout.

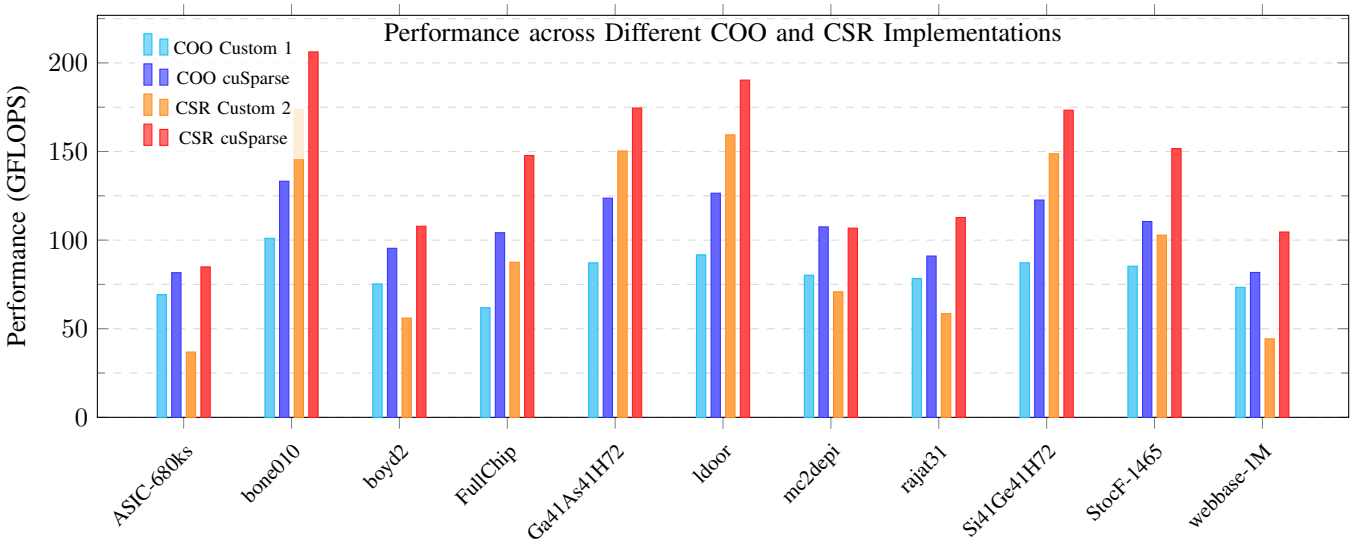

Fig. 3. GFLOPS comparison between various GPU accelerated implementations using the COO and CSR sparse storage layout.

REFERENCES

[1] Chu, Genshen and He, Yuanjie and Dong, Lingyu and Ding, Zhezhao and Chen, Dandan and Bai, He and Wang, Xuesong and Hu, Changjun, "Efficient Algorithm Design of Optimizing SpMV on GPU" New York, NY, USA, 2023.

[2] The Suite Sparse matrix collection, https://sparse.tamu.edu/ .